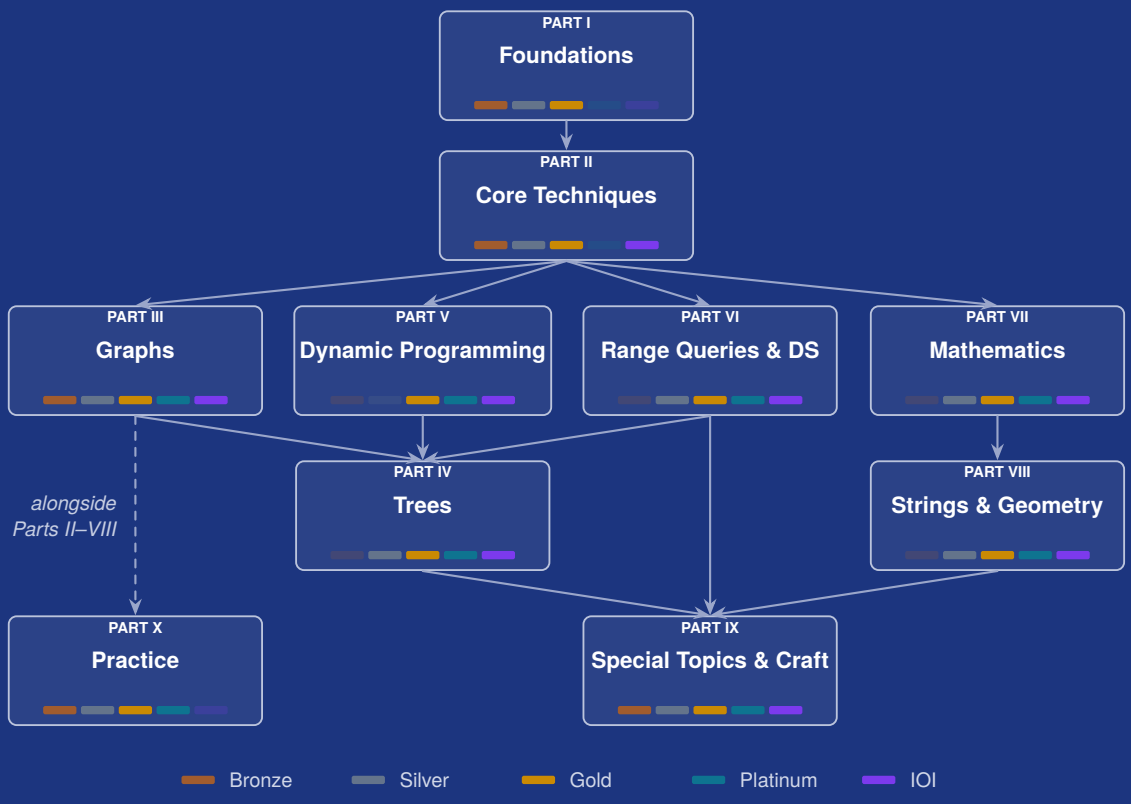
The Competitive Programming Handbook

Data structures, algorithms, patterns and tricks,
from your first loop to IOI gold

SECOND EDITION · 2026



The map of this book: each Part, what it builds on, and the levels it covers.

Kshitish Madbhavi

Companion to the template library in `Desktop/competitive`

The Competitive Programming Handbook

Second edition

About this book

The Competitive Programming Handbook, second edition, by **Kshitish Madbhavi**.

This handbook is the companion to the template library in `Desktop/competitive`. Every technique that has a template shows its keyboard shortcut and snippet prefix next to the explanation, so you can read an idea and have its code at your cursor a second later. The tools around it (CPH integration, the test runner, stress testing, the compiler setup) are described in the companion *User Guide*, `docs/GUIDE.pdf`.

Rebuilding. The book is generated from LaTeX sources in `docs/algorithms/`. Run the VS Code task *CP: Build Docs* or `python tools/cp.py docs`; `python tools/cp.py docs -chapter 13` rebuilds a single chapter for a quick look.

Code. Every program is C++17 and assumes the template (`#define int long long`, `rep`, `all`, `print`, ...). Library templates were checked against brute-force solutions on thousands of random inputs; the worked problems were compiled with the exact template and compiler flags and stress tested.

Typography. Set with pdfLaTeX in TeX Gyre Pagella, TeX Gyre Heros and Source Code Pro. Figures are drawn with TikZ and pgfplots in the Okabe–Ito palette, which stays distinguishable for colour-blind readers; every highlight also differs in line weight, so the figures survive grayscale printing.

Build: October 1, 2026.

Contents

How to Read This Book xiv

I Foundations 1

1 How to Approach a Problem 2

1.1 Anatomy of a problem . . . 2
1.2 A method that works . . . 3
1.3 Thinking tools . . . 7
1.4 When you are stuck . . . 7
1.5 Proving greedy choices . . . 9
1.6 Testing an idea before trusting it . . . 11
1.7 Kinds of problems . . . 11
1.8 How to practise . . . 12

2 Problem-Solving Patterns 14

2.1 Contribution: count each piece once . . . 14
2.2 Fix one thing . . . 16
2.3 Complementary counting . . . 17
2.4 Reverse time and go offline . . . 18
2.5 Events and sweeps . . . 19
2.6 Invariants and colourings . . . 20
2.7 Monovariants and termination . . . 21
2.8 Pigeonhole . . . 22
2.9 The extremal principle . . . 23
2.10 Symmetry and reflection . . . 23
2.11 When the answer is small . . . 24
2.12 Heavy and light . . . 25
2.13 Differences instead of values . . . 26
2.14 Reductions to known problems . . . 27

3 Complexity and Constraints 29

3.1 Counting operations . . . 29

3.2 Big-O notation . . . 30
3.3 What the limits tell you . . . 31
3.4 Analysing loops . . . 32
3.5 Amortized analysis . . . 33
3.6 Recursion . . . 34
3.7 Memory . . . 35
3.8 How fast is an operation, really? . . . 36
3.9 Constant factors that matter . . . 36
3.10 Estimating before coding . . . 37

4 C++ and the STL for Contests **39**
4.1 Input and output . . . 39
4.2 Integer types and overflow . . . 40
4.3 Choosing a container . . . 41
4.4 Sorting and comparators . . . 45
4.5 The algorithm toolbox . . . 46
4.6 Modern C++ that saves time . . . 47
4.7 Policy-based data structures . . . 47
4.8 Randomness . . . 48
4.9 Undefined behaviour and common bugs . . . 48

5 Choosing a Data Structure **50**
5.1 Operations decide . . . 50
5.2 Stacks . . . 51
5.3 Queues and deques . . . 53
5.4 Heaps . . . 54
5.5 Ordered sets and interval sets . . . 56
5.6 Hash maps for counting . . . 57
5.7 Augmenting simple structures . . . 57
5.8 Linked lists in arrays . . . 58

II Core Techniques **61**

6 Simulation and Complete Search **62**
6.1 Simulation . . . 62
6.2 Complete search with loops . . . 64

6.3 Generating subsets . . . 66
6.4 Generating permutations . . . 68
6.5 Backtracking and pruning . . . 69
6.6 Meet in the middle . . . 73
6.7 Fracturing search . . . 75
6.8 Casework and ad hoc problems . . . 76

7 Sorting and Greedy Algorithms 79
7.1 Sorting as a tool . . . 79
7.2 Counting sort and radix sort . . . 81
7.3 Comparators and exchange arguments . . . 82
7.4 Coordinate compression . . . 84
7.5 Greedy algorithms and their proofs . . . 85
7.6 Classic greedy problems . . . 86
7.7 Greedy with heaps and ordered sets . . . 89

8 Binary Search and Unimodal Optimisation 94
8.1 Searching a sorted array . . . 94
8.2 Binary search on a monotone predicate . . . 96
8.3 Binary search on the answer . . . 98
8.4 Real numbers and fractional programming . . . 100
8.5 Ternary search . . . 102
8.6 Parallel binary search . . . 104

9 Prefix Sums, Two Pointers and Sliding Windows 106
9.1 Prefix sums . . . 106
9.2 Difference arrays . . . 108
9.3 Counting subarrays with a hash map . . . 110
9.4 Two pointers . . . 112
9.5 Sliding windows . . . 116
9.6 Monotonic stacks in action . . . 117

10 Bits and Rectangles 120
10.1 Binary numbers and two's complement . . . 120
10.2 Bit tricks . . . 121
10.3 XOR . . . 123

10.4 Bitsets . . . 126
10.5 Gray codes and de Bruijn sequences . . . 127
10.6 Rectangle geometry . . . 130

III Graphs **132**

11 Graph Fundamentals **133**
11.1 Vocabulary . . . 133
11.2 Representing a graph . . . 134
11.3 Depth-first search . . . 135
11.4 Breadth-first search . . . 137
11.5 Grids and flood fill . . . 140
11.6 Bipartite graphs . . . 142
11.7 Finding cycles . . . 143
11.8 Functional graphs . . . 146
11.9 Bidirectional search and A* . . . 147

12 Shortest Paths **151**
12.1 Choosing the algorithm . . . 151
12.2 Relaxation . . . 152
12.3 Dijkstra's algorithm . . . 152
12.4 0–1 BFS . . . 156
12.5 Negative weights: Bellman–Ford . . . 157
12.6 Difference constraints . . . 159
12.7 All pairs: Floyd–Warshall . . . 161
12.8 Shortest and longest paths in DAGs . . . 162

13 Union–Find, Spanning Trees and Topological Order **165**
13.1 Disjoint set union . . . 165
13.2 Minimum spanning trees . . . 168
13.3 Prüfer codes . . . 172
13.4 Topological order . . . 174

14 Connectivity **177**
14.1 Strongly connected components . . . 177
14.2 2-SAT . . . 181

14.3 Bridges and articulation points . . . 183
14.4 Eulerian paths and circuits . . . 186
14.5 Offline dynamic connectivity . . . 188
14.6 Euler's formula and planar graphs . . . 190
14.7 Critical vertices and dominators . . . 191

15 Flows and Matchings **193**
15.1 Maximum flow . . . 193
15.2 Minimum cut . . . 195
15.3 Disjoint paths . . . 197
15.4 Bipartite matching . . . 198
15.5 König, Hall and path covers . . . 199
15.6 The assignment problem . . . 201
15.7 Minimum-cost flow . . . 202
15.8 Flows with lower bounds . . . 202
15.9 Matching in general graphs . . . 203
15.10 Global and all-pairs minimum cuts . . . 203

IV Trees **206**

16 Tree Techniques **207**
16.1 Rooted trees . . . 207
16.2 Diameter and farthest vertices . . . 209
16.3 The Euler tour technique . . . 210
16.4 Binary lifting and the LCA . . . 213
16.5 Offline LCA . . . 216
16.6 DP on trees . . . 218
16.7 Rerooting . . . 220
16.8 Merging child tables . . . 222
16.9 Small-to-large merging . . . 224
16.10 Tree isomorphism . . . 225

17 Tree Decompositions **230**
17.1 Heavy-light decomposition . . . 230
17.2 Centroid decomposition . . . 233
17.3 DSU on tree . . . 238

17.4 Long-path decomposition . . . 241
17.5 Virtual trees . . . 243
17.6 Euler tour trees . . . 245
17.7 Link-cut trees and top trees . . . 246

V Dynamic Programming 249

18 Dynamic Programming Fundamentals 250
18.1 What DP is . . . 250
18.2 The recipe . . . 251
18.3 Coins and sums . . . 253
18.4 Paths in grids . . . 255
18.5 Knapsack . . . 256
18.6 Increasing subsequences . . . 258
18.7 Two sequences: LCS and edit distance . . . 259
18.8 Choosing the state . . . 261
18.9 Reconstruction, memory and debugging . . . 262

19 DP Patterns 265
19.1 Bitmask DP . . . 265
19.2 Interval DP . . . 268
19.3 Digit DP . . . 269
19.4 Boundaries and profiles . . . 270
19.5 Sums over subsets . . . 272
19.6 Scheduling with sorted events . . . 274
19.7 Probability and expectation . . . 275
19.8 Games . . . 278
19.9 Matrix exponentiation . . . 279

20 DP Optimizations 281
20.1 Windows and monotone queues . . . 282
20.2 Divide and conquer optimization . . . 283
20.3 Monotone minima and SMAWK . . . 284
20.4 Knuth's optimization . . . 285
20.5 Convex hull trick . . . 286
20.6 Slope trick . . . 288

20.7 Aliens trick . . . 290
20.8 Convex (min, +) convolution . . . 291
20.9 Bitsets and swapped dimensions . . . 292

VI Range Queries and Data Structures 294

21 Static Range Queries 295
21.1 Sparse table . . . 295
21.2 Disjoint sparse table . . . 297
21.3 Cartesian trees . . . 298
21.4 Square-root decomposition . . . 299
21.5 Mo's algorithm . . . 301

22 Fenwick Trees and Segment Trees 306
22.1 Fenwick trees . . . 306
22.2 Segment trees . . . 310
22.3 Descending a segment tree . . . 310
22.4 Designing a node . . . 311
22.5 Lazy propagation . . . 313
22.6 Counting minimums . . . 314
22.7 Dynamic and persistent segment trees . . . 315
22.8 Segment tree beats . . . 316
22.9 Offline sweeps . . . 317
22.10A staircase for dominance . . . 319

23 Ordered Sets, Treaps and Other Structures 320
23.1 Sets beyond lookups . . . 320
23.2 Order statistics . . . 322
23.3 Treaps . . . 324
23.4 CDQ divide and conquer . . . 327
23.5 Wavelet trees . . . 329
23.6 Hash maps and hashing sets . . . 330

VII Mathematics 334

24 Number Theory 335

24.1 Divisors . . . 335
24.2 GCD and LCM . . . 337
24.3 Primes and sieves . . . 338
24.4 Primality and factoring large numbers . . . 340
24.5 Modular arithmetic . . . 342
24.6 Extended Euclid and linear congruences . . . 344
24.7 Chinese remainder theorem . . . 345
24.8 Euler's totient function . . . 346
24.9 Primitive roots and discrete logarithms . . . 347
24.10 Square roots modulo a prime . . . 349
24.11 Möbius inversion and Dirichlet convolution . . . 350
24.12 Floor-division blocks . . . 351
24.13 Prefix sums of multiplicative functions . . . 352
24.14 The Stern–Brocot tree . . . 354

25 Combinatorics, Probability and Games **357**
25.1 Counting principles . . . 357
25.2 Binomial coefficients . . . 358
25.3 Stars and bars and lattice paths . . . 360
25.4 Catalan numbers and the reflection principle . . . 361
25.5 Inclusion–exclusion . . . 362
25.6 Burnside's lemma . . . 364
25.7 Stirling numbers . . . 365
25.8 Lagrange interpolation . . . 367
25.9 Determinants, spanning trees and paths . . . 368
25.10 Probability and expected value . . . 369
25.11 Impartial games . . . 370

26 Linear Algebra and Polynomials **374**
26.1 Matrices and walks . . . 374
26.2 Gaussian elimination . . . 376
26.3 Linear bases over GF(2) . . . 377
26.4 The fast Fourier transform . . . 379
26.5 The number theoretic transform . . . 382
26.6 Operations on power series . . . 382
26.7 Walsh–Hadamard transforms . . . 385

26.8 Linear recurrences . . . 386

VIII Strings and Geometry **388**

27 Strings **389**

27.1 String hashing . . . 389
27.2 Borders and the prefix function . . . 391
27.3 The Z-function . . . 393
27.4 Tries . . . 394
27.5 Aho–Corasick . . . 395
27.6 Palindromes . . . 396
27.7 Lyndon words and the minimal rotation . . . 398
27.8 Suffix arrays . . . 398
27.9 Suffix automata . . . 401
27.10 Choosing a string tool . . . 402

28 Computational Geometry **404**

28.1 Vectors and the cross product . . . 404
28.2 Segment intersection . . . 405
28.3 Polygons . . . 406
28.4 Convex hull . . . 408
28.5 Rotating calipers . . . 410
28.6 The closest pair of points . . . 411
28.7 Manhattan distances . . . 411
28.8 Sweep lines . . . 413
28.9 Half-plane intersection . . . 414
28.10 The minimum enclosing circle . . . 415

IX Special Topics and Contest Craft **418**

29 Special Topics **327**

29.1 Randomized algorithms . . . 327
29.2 Interactive problems . . . 327
29.3 Communication problems . . . 328
29.4 Bitsets . . . 328
29.5 Matroids . . . 328

29.6 Vectorization and pragmas . . . 329
29.7 Constructive problems . . . 329

30 Contest Craft **330**
30.1 During a contest . . . 330
30.2 Debugging . . . 330
30.3 Recognising the technique . . . 331
30.4 Numbers worth remembering . . . 333
30.5 How to practise . . . 333

X Practice **334**

31 Worked Problems **335**
31.1 The method . . . 335
31.2 Maximum subarray sum . . . 336
31.3 Apartments . . . 338
31.4 Movie festival . . . 339
31.5 Factory machines . . . 340
31.6 Subarray sums . . . 342
31.7 Counting rooms . . . 343
31.8 Message route . . . 345
31.9 Shortest routes . . . 346
31.10 Road construction . . . 348
31.11 Coin combinations . . . 349
31.12 Book shop . . . 350
31.13 Longest increasing subsequence . . . 351
31.14 Subordinates . . . 353
31.15 Tree diameter . . . 354
31.16 Range sum queries with updates . . . 356
31.17 String matching . . . 357
31.18 Teleporters . . . 358
31.19 Where to go next . . . 359

A Reference Tables **361**
A.1 Complexity of the standard algorithms . . . 361
A.2 Formula sheet . . . 362

A.3 Where the templates are 362

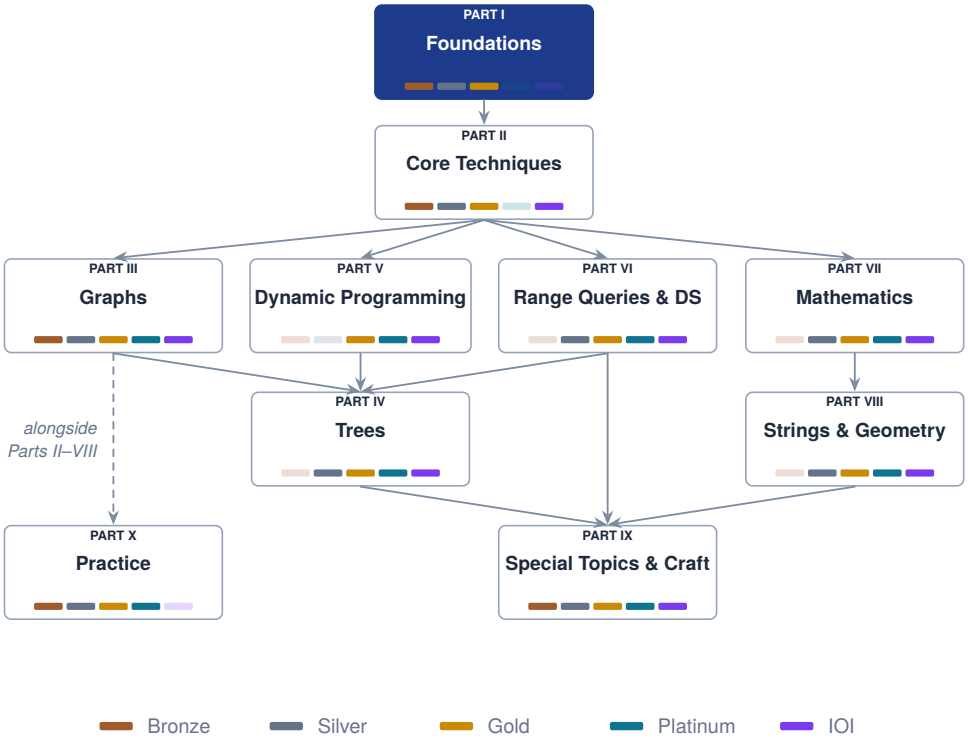
B Template Index **363**

Index **369**

PART I

Foundations

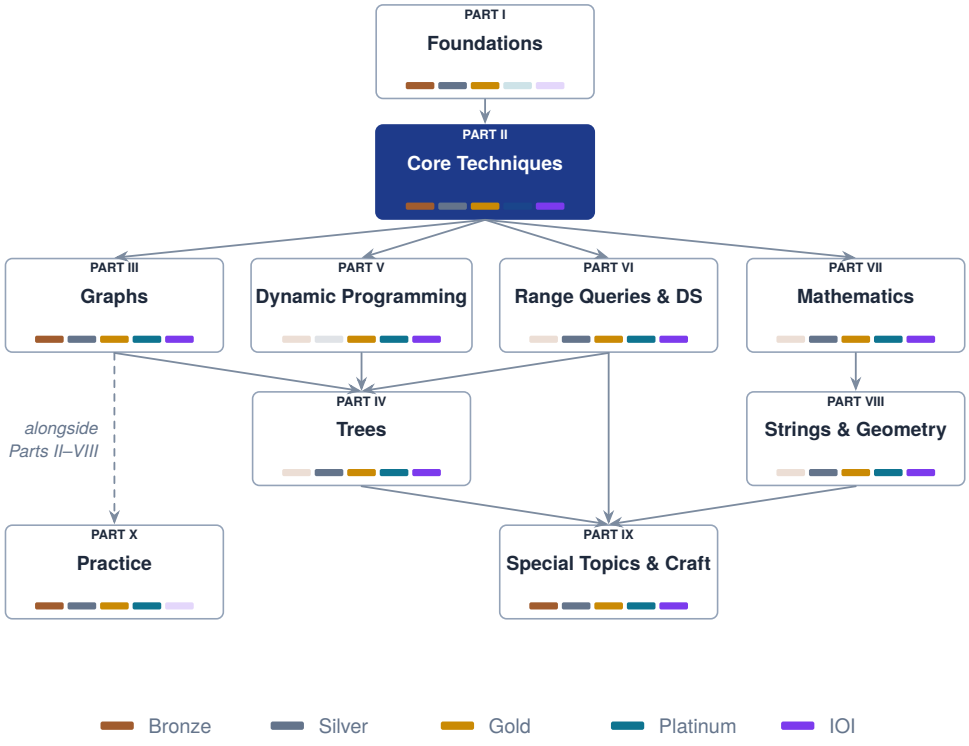
How to attack a problem, how to know whether an idea is fast enough, the C++ you need, and the data structures every solution is built from.



PART II

Core Techniques

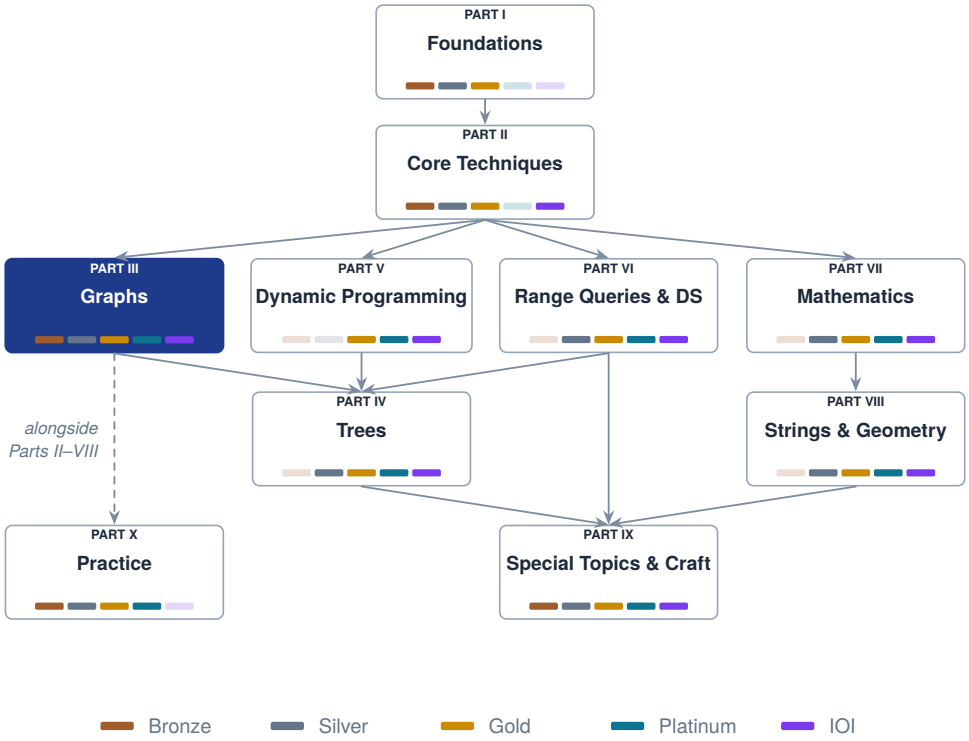
The techniques that solve most Bronze and Silver problems and appear inside nearly every harder one: exhaustive search, sorting and greedy choices, binary search, prefix sums, two pointers and bit manipulation.



PART III

Graphs

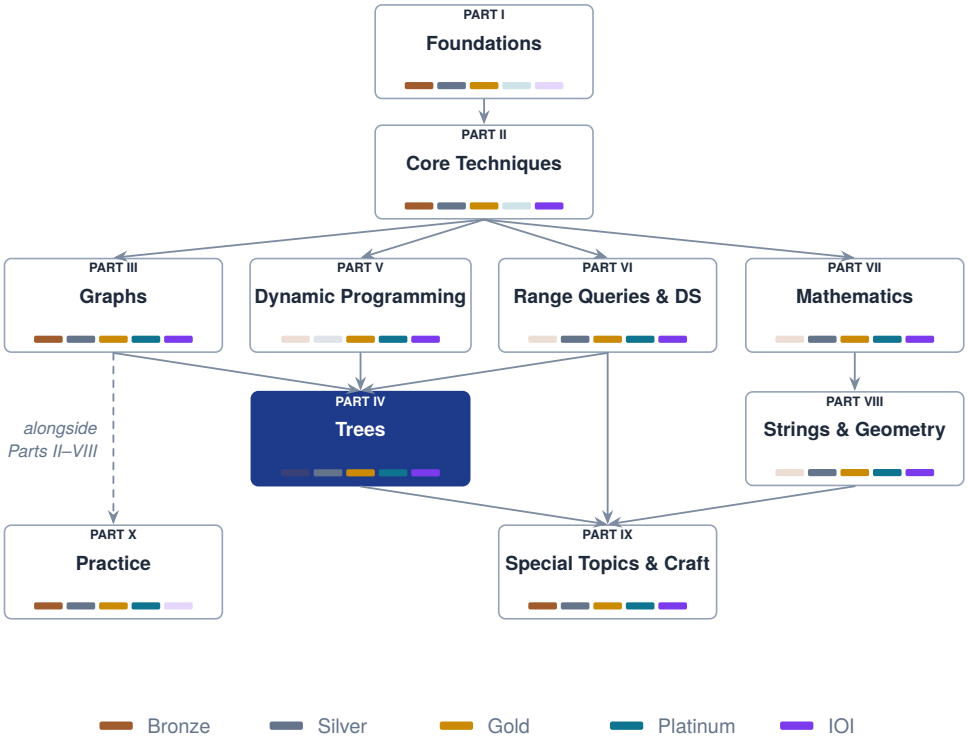
Traversals, shortest paths, spanning trees, connectivity and flows: modelling a problem as a graph and choosing the right algorithm for it.



PART IV

Trees

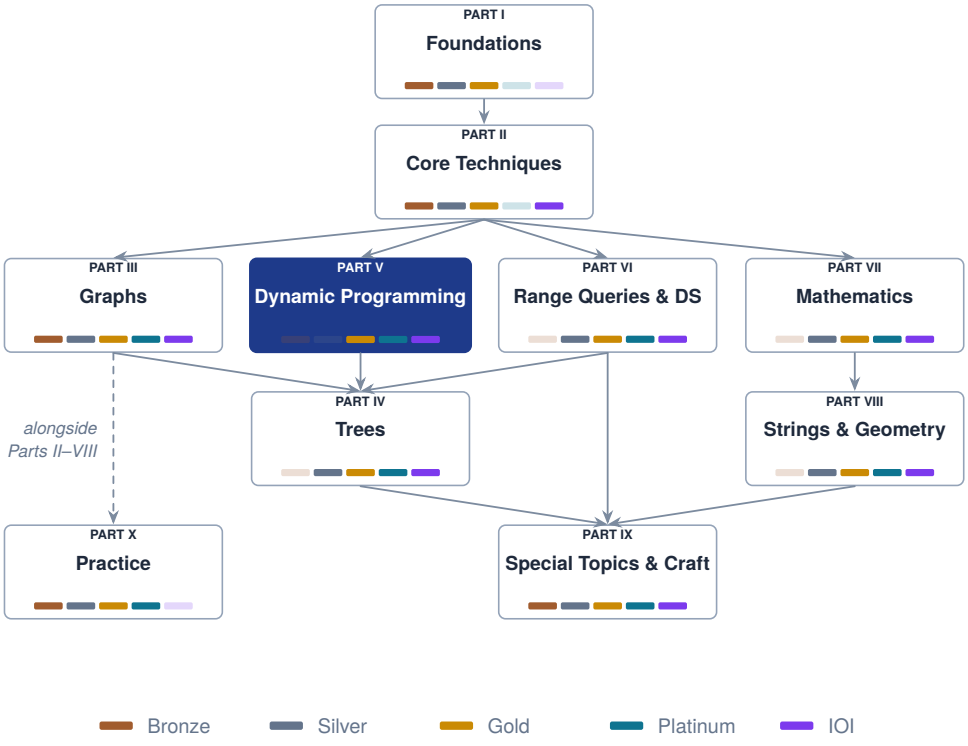
Rooted trees, the Euler tour, binary lifting, DP on trees, and the decompositions (heavy-light, centroid, virtual trees) that turn tree queries into array queries.



PART V

Dynamic Programming

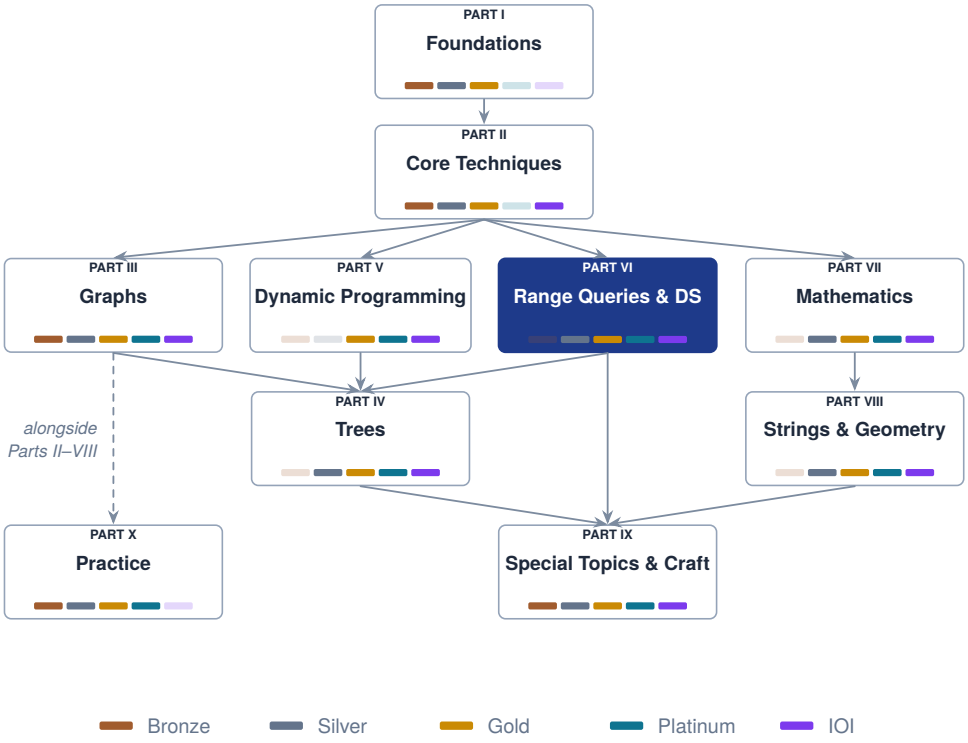
From the first recurrence to the optimizations that decide olympiad medals: knapsack, subsequences, bitmask, range, digit and profile DP, convex hull trick, divide and conquer, Knuth, Aliens and slope trick.



PART VI

Range Queries and Data Structures

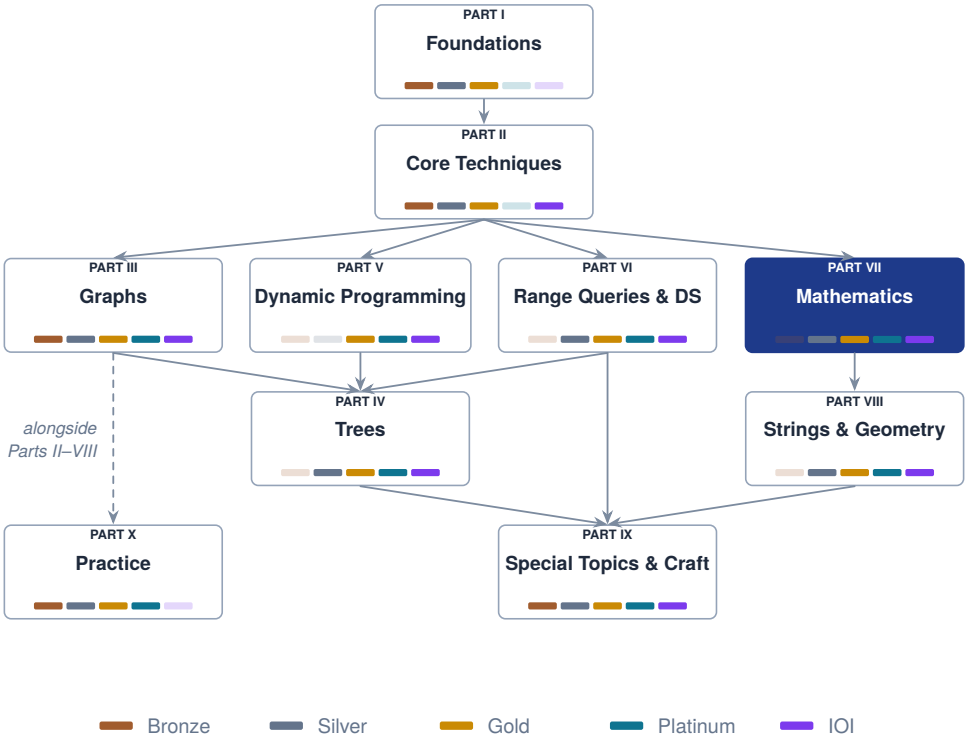
Answering questions about ranges of an array, with and without updates: sparse tables, Fenwick and segment trees in all their forms, square-root techniques, treaps and offline methods.



PART VII

Mathematics

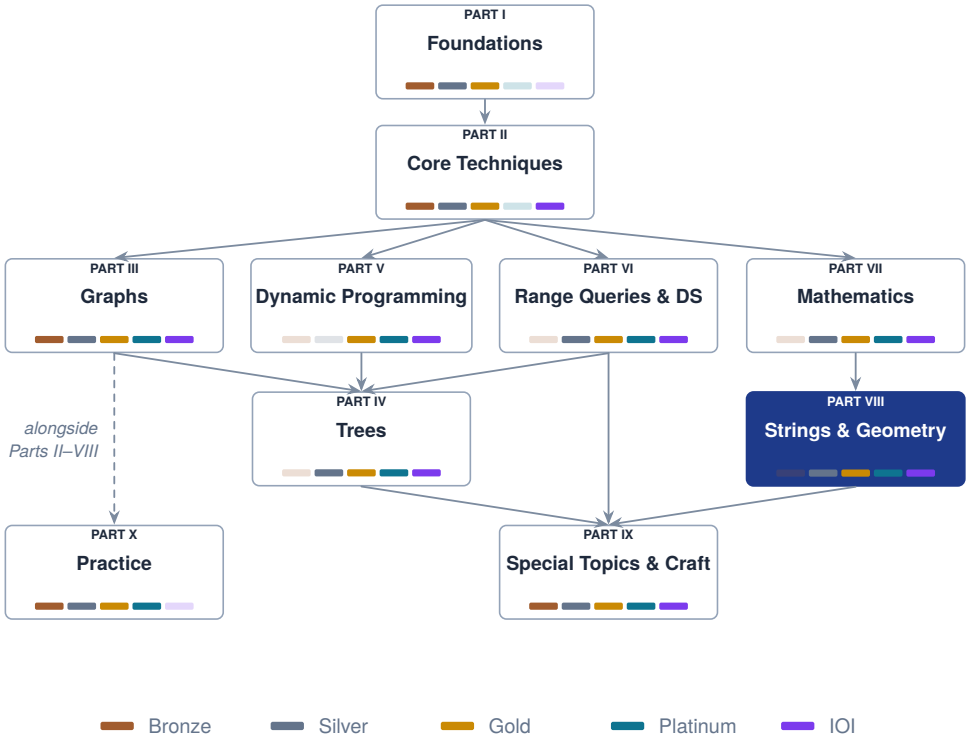
Number theory, modular arithmetic, combinatorics, linear algebra and polynomials: the mathematics that appears in contests, with the algorithms that compute it.



PART VIII

Strings and Geometry

Searching, hashing and suffix structures for strings; vectors, hulls and sweeps for geometry.



CHAPTER 28

Computational Geometry

Geometry problems are mostly a few robust primitives used with care. The golden rule: **stay in integers whenever the input is integral**. Orientation tests, areas, convex hulls and closest pairs need only additions, multiplications and comparisons, and then they are exact. Floating point enters only with circles, angles and intersection points, and there it needs tolerances and a healthy suspicion. This chapter builds the primitives, the classic algorithms on top of them (hulls, rotating calipers, closest pairs, sweeps), and two randomised and incremental gems: half-plane intersection and the minimum enclosing circle.

IN THIS CHAPTER GOLD PLATINUM IOI / ADVANCED

- compute orientations, segment intersections, areas and lattice-point counts exactly with cross products;
- build convex hulls with the monotone chain, and measure them with rotating calipers;
- find the closest pair by a sweep, and handle Manhattan distances by rotating the plane;
- design sweep-line algorithms around events and an active set;
- intersect half-planes, and find the minimum enclosing circle in expected linear time.

Before you start: Chapter 4 (integer overflow) and Chapter 22 (Fenwick trees for sweeps).

28.1 Vectors and the cross product

GOLD

A point is a vector (x, y), and two products of vectors do almost all the work:

- the **dot product** $a \cdot b = a_x b_x + a_y b_y = |a|\,|b| \cos\theta$ is positive when the angle between a and b is acute, zero when they are perpendicular, negative when it is obtuse;
- the **cross product** $a \times b = a_x b_y - a_y b_x = |a|\,|b| \sin\theta$ is positive when b is counter-clockwise from a, negative when it is clockwise, zero when they are collinear; $|a \times b|$ is the area of the parallelogram they span (Figure 28.1).

The **orientation** of three points, $\text{orient}(a, b, c) = \text{sign}\big((b - a) \times (c - a)\big)$, says whether the path $a \to b \to c$ turns left (+1), right (−1), or goes straight (0). It is the single most important primitive of the chapter.

</> Template: Geometry primitives Ctrl+Alt+T, Shift+3 prefix `cppoint`

Example: Point Location Test (CSES)

For each of 10^5 tests, decide whether a point lies to the left of, to the right of, or on the line through two other points (coordinates up to 10^9).

Solution. One orientation per test: 0.14 s.

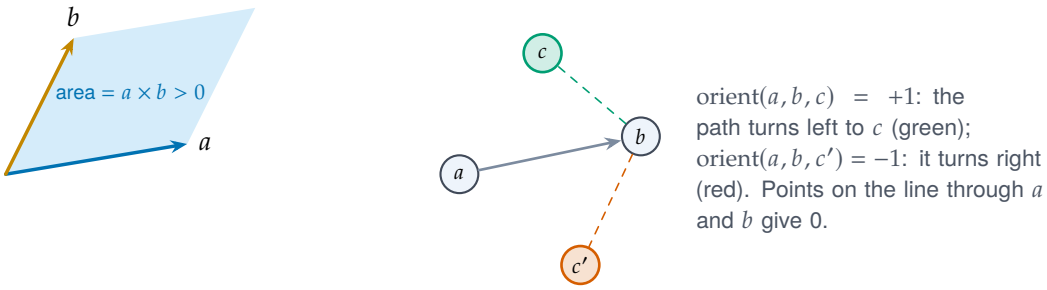


Figure 28.1: The cross product is a signed area, and its sign is an orientation.

```
void solve() {  // Point, orient (cppoint) are pasted above solve()
    int t;
    cin >> t;
    while (t--) {
        Point a, b, c;
        cin >> a >> b >> c;
        int o = orient(a, b, c);  // the sign of (b - a) x (c - a)
        print(o > 0 ? "LEFT" : o < 0 ? "RIGHT" : "TOUCH");
    }
}
```

Precision and overflow Coordinates up to 10^9 give differences up to $2 \cdot 10^9$ and cross products up to $8 \cdot 10^{18}$: they fit in a `long long`, barely. Anything that multiplies three coordinates does not; use `__int128` there. When floating point is unavoidable (circles, angles, intersection points), use `long double`, compare with a tolerance such as 10^{-9} scaled to the magnitudes involved, and replace `sqrt` and `atan2` by comparisons of squared lengths and cross products wherever possible.

Sorting by angle without atan2 To sort vectors by polar angle exactly, split them into two half-planes, the upper one ($y > 0$, or $y = 0$ and $x > 0$) and the rest; compare the halves first, and within a half put a before b when $a \times b > 0$. The comparison is integer-exact, it is the basis of angular sweeps ("the most points inside a 90° wedge"), and it also sorts the half-planes of Section 28.9.

28.2 Segment intersection

GOLD

Segments ab and cd cross properly exactly when c and d lie strictly on opposite sides of the line ab *and* a and b lie strictly on opposite sides of the line cd: $\text{orient}(a,b,c) \cdot \text{orient}(a,b,d) < 0$ and $\text{orient}(c,d,a) \cdot \text{orient}(c,d,b) < 0$. Every other way of sharing a point puts an endpoint of one segment on the other segment, which is checked directly; collinear segments that do not overlap must be rejected even though all four orientations are 0 (Figure 28.2).

</> Template: Segment intersection Ctrl+Alt+Y, 7 prefix `cpseginter`

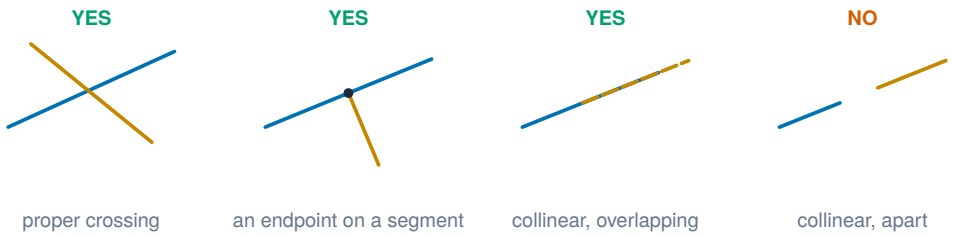


Figure 28.2: The cases of segment intersection. Only the first is decided by the signs alone; the others need "does an endpoint lie on the other segment", and the last shows why collinearity alone is not enough.

Example: Line Segment Intersection (CSES)

For each of 10^5 pairs of segments with integer coordinates up to 10^9, decide whether they share a point.

Solution. Four orientations and four "point on segment" tests per pair, all exact: 0.17 s.

```
void solve() {  // Point (cppoint), segments_intersect (cpseginter) pasted above
    int t;
    cin >> t;
    while (t--) {
        Point a, b, c, d;
        cin >> a >> b >> c >> d;
        yn(segments_intersect(a, b, c, d));
    }
}
```

When the intersection *point* is needed, solve $a + t(b-a) = c + u(d-c)$: with $r = b - a$ and $s = d - c$, $t = \frac{(c-a)\times s}{r\times s}$. The point has rational coordinates, so keep the numerator and the denominator as integers if it must be compared exactly, and only convert to floating point for printing.

28.3 Polygons

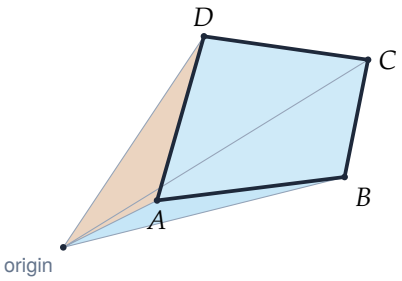
GOLD

Area. For a polygon with vertices $p_0, \ldots, p_{n-1}$ in order, the **shoelace formula** gives twice its signed area:

$$2A = \sum_{i=0}^{n-1} p_i \times p_{i+1} \qquad (p_n = p_0),$$

positive for counter-clockwise order. Each term is twice the signed area of the triangle formed by the origin and one edge; triangles of edges seen counter-clockwise from the origin count positively and the others negatively, and everything outside the polygon cancels (Figure 28.3).

Template: Polygon: area, point inside, lattice points Ctrl+Alt+Y, 8 prefix `cppolygon`



The polygon $ABCD$ (counter-clockwise). The triangles OAB, OBC and OCD (blue) have positive signed area, ODA (red) negative; the red one exactly cancels the part of the blue ones that lies outside the polygon.

Figure 28.3: The shoelace formula as a sum of signed triangles.

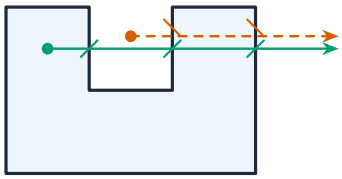
Example: Polygon Area (CSES)

Print twice the area of a polygon given by its vertices, with coordinates up to 10^9.

Solution. The absolute value of the shoelace sum: twice the area is an integer, so print it without dividing.

```
void solve() {  // Point (cppoint), area2 (cppolygon) are pasted above solve()
    int n;
    cin >> n;
    vector<Point> poly(n);
    for (auto& p : poly) cin >> p;
    print(llabs(area2(poly)));  // the shoelace sum is twice the signed area
}
```

Point in polygon. Cast a ray from the point to the right and count the edges it crosses: an odd count means inside (Figure 28.4). Check "on the boundary" first, and count an edge only when the point's y lies in the half-open range $[\min(y_1, y_2), \max(y_1, y_2))$ of the edge, so that a ray through a vertex is counted once, and a horizontal edge never.



The green point is inside: its ray crosses three edges. The red point, in the notch, is outside: its ray crosses two. Only the parity matters, whatever the shape.

Figure 28.4: Ray casting: an odd number of crossings means inside.

Example: Point in Polygon (CSES)

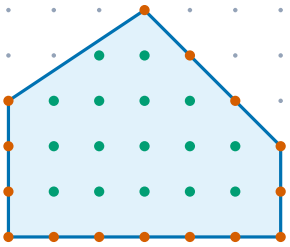
For $m \le 1000$ points and a polygon with $n \le 1000$ vertices, print INSIDE, OUTSIDE or BOUNDARY for every point.

Solution. Ray casting in $O(n)$ per point.

```
void solve() {  // Point (cppoint), in_polygon (cppolygon) pasted above
    int n, m;
    cin >> n >> m;
    vector<Point> poly(n);
    for (auto& p : poly) cin >> p;
    while (m--) {
```

```
        Point q;
        cin >> q;
        int w = in_polygon(poly, q);  // parity of ray crossings
        print(w > 0 ? "INSIDE" : w < 0 ? "OUTSIDE" : "BOUNDARY");
    }
}
```

Lattice points. Pick's theorem: a simple polygon whose vertices are lattice points has area $A = I + \frac{B}{2} - 1$, where I counts the lattice points strictly inside and B those on the boundary (Figure 28.5). An edge from p to q contains $\gcd(|q_x - p_x|, |q_y - p_y|)$ lattice points, counting q but not p, so B is a sum of gcds, and $I = (2A - B + 2)/2$ follows without enumerating any point.



The pentagon $(0,0), (6,0), (6,2), (3,5), (0,3)$ has $2A = 45$. Its boundary carries $B = 15$ lattice points (red), so $I = (45 - 15 + 2)/2 = 16$ lie inside (green): $A = 16 + 7.5 - 1 = 22.5$.

Figure 28.5: Pick's theorem counts lattice points from the area and the boundary.

Example: Polygon Lattice Points (CSES)

Count the lattice points strictly inside and on the boundary of a polygon with $n \le 10^5$ vertices and coordinates up to 10^9.

Solution. Pick's theorem with the shoelace area and a gcd per edge: 0.09 s.

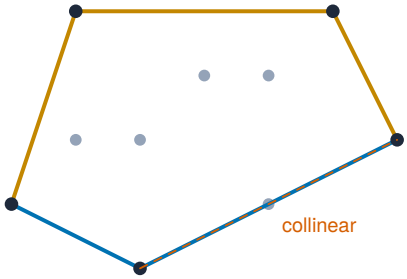
```
void solve() {  // Point, area2, boundary_points (cppoint, cppolygon) pasted
    int n;
    cin >> n;
    vector<Point> poly(n);
    for (auto& p : poly) cin >> p;
    ll twice = llabs(area2(poly)), b = boundary_points(poly);
    print((twice - b + 2) / 2, b);  // Pick's theorem: A = I + B / 2 - 1
}
```

28.4 Convex hull

PLATINUM

The **convex hull** of a point set is the smallest convex polygon that contains it: a rubber band stretched around the points. Andrew's **monotone chain** sorts the points by x (then y) and builds the lower hull from left to right: append each point, but first pop the last point of the chain while the last two chain points and the new one do not turn left. The upper hull is the same scan from right to left, and the two chains together form the hull (Figure 28.6). A popped point lies inside the triangle formed by its neighbours and a point below it, so it can never return, and every point is pushed and popped at most once per chain: $O(n)$ after the $O(n \log n)$ sort.

</> Template: Convex hull Ctrl+Alt+T, Shift+V prefix `cphull`



Lower hull (blue) from left to right, upper hull (orange) from right to left. The point $(4, 1)$ lies on the segment from $(2, 0)$ to $(6, 2)$: the template pops it, which gives the hull's vertices; popping only on strict right turns keeps it, which lists every point on the boundary.

Figure 28.6: The monotone chain on ten points: lower and upper hull.

Example: Convex Hull (CSES)

Print all of the $n \le 2 \cdot 10^5$ points that lie on the boundary of the convex hull, collinear points included.

Solution. The monotone chain, popping only on strict right turns so that collinear points stay. If all the points are collinear, both chains list all of them; sorting and removing duplicates at the end handles that case and the two shared endpoints. 0.13 s.

```
void solve() {
    int n;
    cin >> n;
    vector<pll> p(n);
    for (auto& [x, y] : p) cin >> x >> y;
    sort(all(p));
    auto cross = [](pll o, pll a, pll b) {  // > 0: o -> a -> b turns left
        ll ax = a.first - o.first, ay = a.second - o.second;
        return ax * (b.second - o.second) - ay * (b.first - o.first);
    };
    vector<pll> h;
    rep(pass, 2) {  // lower hull left to right, then upper hull back
        int b0 = sz(h);  // where this chain starts
        for (auto q : p) {
            while (sz(h) >= b0 + 2 && cross(h[sz(h) - 2], h.back(), q) < 0)
                h.pop_back();  // pop only right turns: collinear points stay
            h.push_back(q);
        }
        reverse(all(p));
    }
    sort(all(h));  // the chains share their ends; collinear inputs repeat too
    h.erase(unique(all(h)), h.end());
    print(sz(h));
    for (auto [x, y] : h) print(x, y);
}
```

What to do with a hull

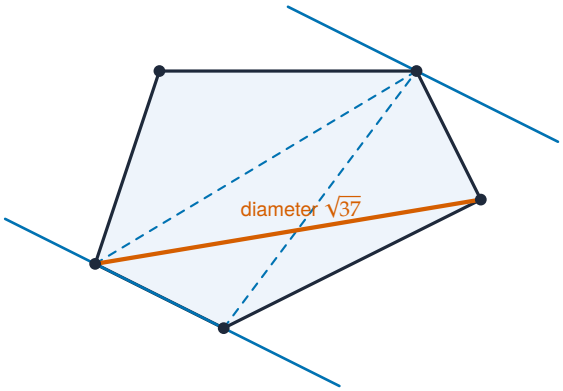
- A linear function $ax + by$ is maximised at a hull vertex; with the hull's edge directions sorted, a binary search finds it in $O(\log h)$ per query. The convex hull trick of Section 20.5 is this idea for lines.

- A point is inside a convex polygon iff it lies in the wedge at vertex 0 found by binary search, on the inner side of that wedge's edge: $O(\log n)$.
- The farthest pair, the width, and the smallest enclosing rectangle are found by rotating calipers (next section).

28.5 Rotating calipers

IOI / ADVANCED

The two farthest points of a set are both vertices of its convex hull, and they are *antipodal*: some pair of parallel lines through them encloses the whole hull. **Rotating calipers** enumerate all antipodal pairs in $O(h)$: for every edge of the hull, in order, advance a second pointer to the vertex farthest from that edge's line (the area of the triangle grows and then shrinks, so the pointer only moves forward), and test the distances from the edge's endpoints to that vertex (Figure 28.7). The same walk gives the width of the polygon, the smallest distance between two parallel lines enclosing it, which always has one line flush with an edge.



For the edge from $(0,1)$ to $(2,0)$ the farthest vertex is $(5,4)$, and the two blue lines are calipers enclosing the hull. Testing the pairs (dashed) for every edge finds the farthest pair, $(0,1)$ and $(6,2)$ (red).

Figure 28.7: Rotating calipers on a convex pentagon.

</> Template: Rotating calipers Ctrl+Alt+U, 5 prefix `cpcalipers`

Example: Farthest pair

Find the largest squared distance between two of $n \le 2 \cdot 10^5$ points with coordinates up to 10^9.

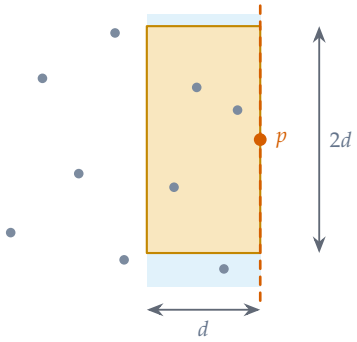
Solution. The convex hull, then rotating calipers over its vertices: 0.13 s, against $O(n^2)$ for all pairs.

```
void solve() {  // convex_hull (cphull), hull_diameter2 (cpcalipers) pasted
    int n;
    cin >> n;
    vector<pll> p(n);
    for (auto& [x, y] : p) cin >> x >> y;
    print(hull_diameter2(convex_hull(p)));  // both ends are hull vertices
}
```

28.6 The closest pair of points

PLATINUM

Sort the points by x and sweep from left to right, keeping the best squared distance d^2 found so far and, in a set ordered by y, the points whose x is within d of the sweep line. A new point p can only improve the answer with points of that set whose y lies in $[p_y - d, p_y + d]$: a $d \times 2d$ box (Figure 28.8). The points in the box are pairwise at least d apart, so at most 8 of them fit, and each step costs $O(\log n)$: the whole sweep is $O(n \log n)$, with exact integer distances.



Only the points left of the sweep line by less than d (blue strip) are kept, and of those only the ones in the orange box can be closer to p than d. Points at mutual distance at least d fit into a $d \times 2d$ box at most eight at a time.

Figure 28.8: The closest-pair sweep checks a constant number of candidates per point.

</> Template: Closest pair Ctrl+Alt+Y, 9 prefix `cpclosest`

Example: Minimum Euclidean Distance (CSES)

Print the smallest squared distance between two of $n \le 2 \cdot 10^5$ points.

Solution. The sweep, in exact integers: 0.13 s.

```
void solve() {  // Point (cppoint), closest_pair (cpclosest) pasted above
    int n;
    cin >> n;
    vector<Point> p(n);
    for (auto& q : p) cin >> q;
    print(closest_pair(p));  // squared, so it stays exact
}
```

28.7 Manhattan distances

GOLD

The Manhattan distance $|x_1 - x_2| + |y_1 - y_2|$ becomes much easier after rotating the plane by 45°. With $u = x + y$ and $v = x - y$,

$$|x_1 - x_2| + |y_1 - y_2| = \max(|u_1 - u_2|, |v_1 - v_2|),$$

because $|a| + |b| = \max(|a + b|, |a - b|)$. The diamond of points within Manhattan distance r becomes an axis-parallel square (Figure 28.9), and the two coordinates separate: the farthest pair, nearest-neighbour queries and "all points within distance r" become independent questions about u and v.

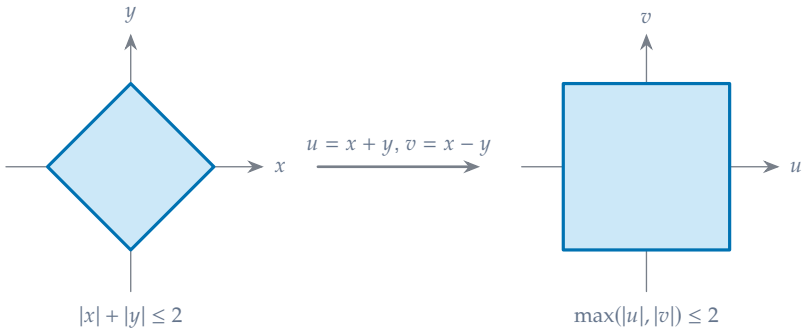


Figure 28.9: Rotating by 45° turns Manhattan balls into squares.

Example: Maximum Manhattan Distances (CSES)

Points arrive one at a time ($n \le 2 \cdot 10^5$); after each, print the largest Manhattan distance between two points so far.

Solution. In rotated coordinates the answer is $\max(u_{\max} - u_{\min}, v_{\max} - v_{\min})$: four running extremes. 0.14 s.

```
void solve() {
    int n;
    cin >> n;
    ll umin = INF, umax = -INF, vmin = INF, vmax = -INF;
    rep(i, n) {
        ll x, y;
        cin >> x >> y;
        ll u = x + y, v = x - y;  // |dx| + |dy| = max(|du|, |dv|)
        chmin(umin, u), chmax(umax, u), chmin(vmin, v), chmax(vmax, v);
        print(max(umax - umin, vmax - vmin));
    }
}
```

Sums of distances. The *sum* of Manhattan distances over all pairs separates by axis without any rotation: sort the x-coordinates, and the i-th smallest contributes x_i for each of the i smaller ones minus their sum, the contribution technique of Chapter 2.

Example: All Manhattan Distances (CSES)

Sum the Manhattan distances between all pairs of $n \le 2 \cdot 10^5$ points with coordinates up to 10^9 in absolute value.

Solution. Sort each axis and add up the contributions. The total reaches $2.7 \cdot 10^{19}$ on random inputs at the limits, more than a `long long` holds, so the sum is kept in `__int128` and printed by hand. 0.15 s.

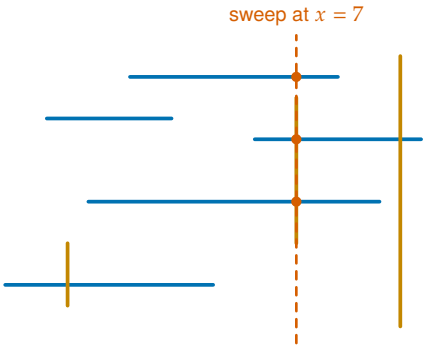
```
void solve() {
    int n;
    cin >> n;
    vi xs(n), ys(n);
    rep(i, n) cin >> xs[i] >> ys[i];
```

```
    __int128 total = 0;  // can exceed 64 bits: up to about 4e19
    for (vi* c : {&xs, &ys}) {  // |dx| + |dy|: the two axes add up separately
        sort(all(*c));
        __int128 pre = 0;  // the sum of the smaller coordinates
        rep(i, n) total += (__int128)(*c)[i] * i - pre, pre += (*c)[i];
    }
    string s;
    for (; total > 0; total /= 10) s += char('0' + total % 10);
    if (s.empty()) s = "0";
    reverse(all(s));
    print(s);
}
```

28.8 Sweep lines

PLATINUM

A sweep line moves across the plane and processes *events*, such as segment endpoints, points and queries, in order of position, while a data structure holds the *active* objects that the line currently crosses. It turns a two-dimensional problem into a sequence of one-dimensional ones. Designing a sweep means answering three questions: what are the events, in which order do events at the same position happen, and what must the active set support? Section 22.6 computed the area of a union of rectangles this way, with a segment tree as the active set.



At $x = 7$ the active horizontals (crossing the line) are at heights 3, 4.5 and 6. The vertical segment from 2 to 5.5 meets the two whose height lies in $[2, 5.5]$: a range count in a Fenwick tree over the heights.

Figure 28.10: Counting intersections of horizontal and vertical segments by a sweep.

Example: Intersection Points (CSES)

$n \le 10^5$ horizontal and vertical segments with coordinates up to 10^6 are given, and parallel segments never touch. Count the points where a horizontal and a vertical segment meet.

Solution. Sweep over x. A horizontal segment becomes active at its left end and inactive at its right end; a vertical segment at x asks how many active horizontals have their height in its y-range (Figure 28.10). At equal x, insertions come before queries and queries before removals, so that touching endpoints count. 0.12 s.

```
void solve() {
    int n;
    cin >> n;
    const ll OFF = 1000001, N = 2 * OFF + 1;  // y + OFF lies in [1, N]
    vector<array<ll, 4>> ev;  // (x, type, y1, y2): 0 add, 1 query, 2 remove
```

```
    rep(i, n) {
        ll x1, y1, x2, y2;
        cin >> x1 >> y1 >> x2 >> y2;
        if (y1 == y2) {  // horizontal: alive for x in [min, max]
            ev.push_back({min(x1, x2), 0, y1, y1});
            ev.push_back({max(x1, x2), 2, y1, y1});
        } else ev.push_back({x1, 1, min(y1, y2), max(y1, y2)});
    }
    sort(all(ev));  // at equal x: insert, then query, then remove
    vi bit(N + 1);
    auto add = [&](ll y, ll v) {
        for (ll i = y + OFF; i <= N; i += i & -i) bit[i] += v;
    };
    auto pre = [&](ll y) {  // active horizontals at height <= y
        ll s = 0;
        for (ll i = y + OFF; i > 0; i -= i & -i) s += bit[i];
        return s;
    };
    ll ans = 0;
    for (auto [x, type, a, b] : ev) {
        if (type == 0) add(a, 1);
        else if (type == 2) add(a, -1);
        else ans += pre(b) - pre(a - 1);
    }
    print(ans);
}
```

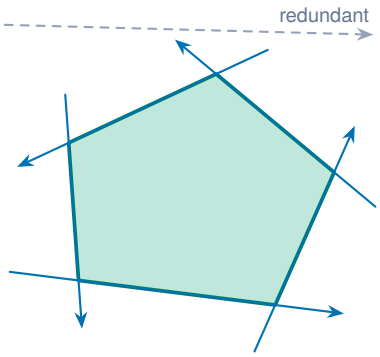
How to recognize it Many intervals, rectangles or segments, and a question about their overlaps, unions or intersections; "for each point, how many rectangles contain it"; "the largest number of overlapping intervals" (a one-dimensional sweep over +1 and −1 events). Sort the events, fix the tie-breaking order, and choose the active set's structure: a counter, a multiset, a Fenwick tree, or a segment tree.

28.9 Half-plane intersection

IOI / ADVANCED

A directed line keeps the half-plane on its left. The intersection of n half-planes is a convex polygon (possibly empty or unbounded), and it is computed in $O(n \log n)$ much like a convex hull: sort the lines by direction angle, then add them in that order to a deque. A new line makes the last line in the deque redundant when the intersection point of the last two lines lies outside the new half-plane, so pop from the back while that happens; the same check at the front removes lines that the new one cuts off from the other end. Of two lines with the same direction only the stricter one is kept. A final cleanup between the two ends of the deque closes the polygon (Figure 28.11). Adding a bounding box of four lines keeps the result bounded.

</> Template: Half-plane intersection Ctrl+Alt+U, 4 prefix `cphalfplane`



Six half-planes, each to the left of its directed line. Five of them bound the intersection (green); the dashed one contains it entirely and is dropped when the deque is cleaned up.

Figure 28.11: Half-plane intersection: the lines sorted by angle form the boundary in order.

Example: Half-plane intersection area

Print the area of the intersection of $n \le 10^5$ half-planes (each to the left of a directed line through two integer points) inside the square $[-10^6, 10^6]^2$.

Solution. The deque algorithm with the square's four sides added, then the shoelace formula. With 10^5 tangent lines of a circle, which all survive, it takes 0.83 s, mostly for the angles and the long-double intersections.

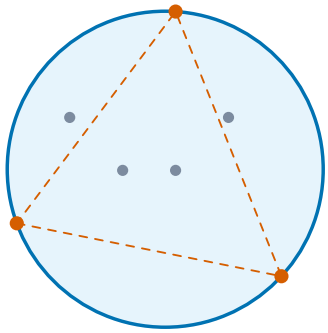
```
void solve() {  // half_plane_intersection (cphalfplane) is pasted above solve()
    int n;
    cin >> n;
    vector<array<ld, 4>> hs(n);
    for (auto& [ax, ay, bx, by] : hs) cin >> ax >> ay >> bx >> by;
    auto poly = half_plane_intersection(hs, 1e6);  // keeps the left of a -> b
    ld area = 0;  // the shoelace formula on the resulting convex polygon
    rep(i, sz(poly)) {
        auto [x1, y1] = poly[i];
        auto [x2, y2] = poly[(i + 1) % sz(poly)];
        area += x1 * y2 - x2 * y1;
    }
    cout << fixed << setprecision(6) << area / 2 << '\n';
}
```

28.10 The minimum enclosing circle

IOI / ADVANCED

The smallest circle that contains n given points is unique, and it is determined by two points on its boundary (a diameter) or by three. **Welzl's algorithm**, in its iterative form, shuffles the points and maintains the smallest circle of the points seen so far. When a new point p_i falls outside, the new circle must pass through p_i; rebuild it from the earlier points with p_i fixed on the boundary, by the same procedure one level down, and when a second point p_j falls outside, with both p_i and p_j on the boundary; at the third level a circle through three points is determined (Figure 28.12).

The rebuilds look expensive, but in random order they are rare. By *backwards analysis*, the circle of the first i points changes when p_i is added only if p_i is one of the at most three points that define it, which happens with probability at most $3/i$; a rebuild costs $O(i)$, so its expected cost is $O(1)$ per point, and the same argument one level down keeps the whole algorithm at expected $O(n)$.



Seven points; the smallest enclosing circle passes through the three red ones, the circumcircle of their triangle, which is acute. Had the triangle been obtuse, the circle would instead have the longest side as a diameter and pass through only two points.

Figure 28.12: The minimum enclosing circle is determined by two or three of the points.

</> Template: Minimum enclosing circle Ctrl+Alt+U, Shift+M prefix `cpmincircle`

Example: Minimum enclosing circle

Print the centre and the radius of the smallest circle containing $n \le 10^5$ points with integer coordinates up to 10^6.

Solution. Welzl's algorithm after a shuffle: 0.51 s, almost all of it in `hypotl`. Comparing squared distances instead would save most of that.

```
void solve() {  // min_enclosing_circle (cpmincircle) is pasted above solve()
    int n;
    cin >> n;
    vector<pair<ld, ld>> p(n);
    for (auto& [x, y] : p) cin >> x >> y;
    auto [cx, cy, r] = min_enclosing_circle(p);  // expected O(n), shuffled
    cout << fixed << setprecision(6) << cx << ' ' << cy << ' ' << r << '\n';
}
```

Shuffle, or be hacked Without the random shuffle, Welzl's algorithm is quadratic on inputs sorted by distance from the centre, and such inputs are easy to construct. Shuffle with a seeded generator (Section 4.8), never with the default seed.

Chapter summary

- The cross product gives signed areas and orientations; stay in integers, and mind that cross products of 10^9-sized coordinates nearly fill a `long long`.
- Segment intersection is four orientations plus endpoint-on-segment checks; the shoelace formula gives areas; ray casting decides point in polygon; Pick's theorem counts lattice points from the area and the boundary.
- The monotone chain builds the convex hull in $O(n \log n)$; rotating calipers find the farthest pair and the width in $O(h)$.
- The closest pair falls to a sweep with a y-ordered set; rotating by 45° turns Manhattan distances into maxima of two independent coordinates.
- A sweep line needs its events, a tie-breaking order and an active set.
- Half-planes intersect in $O(n \log n)$ with a deque over lines sorted by angle; the minimum enclosing circle takes expected $O(n)$ after a random shuffle.

Practice problems (CSES Problem Set)

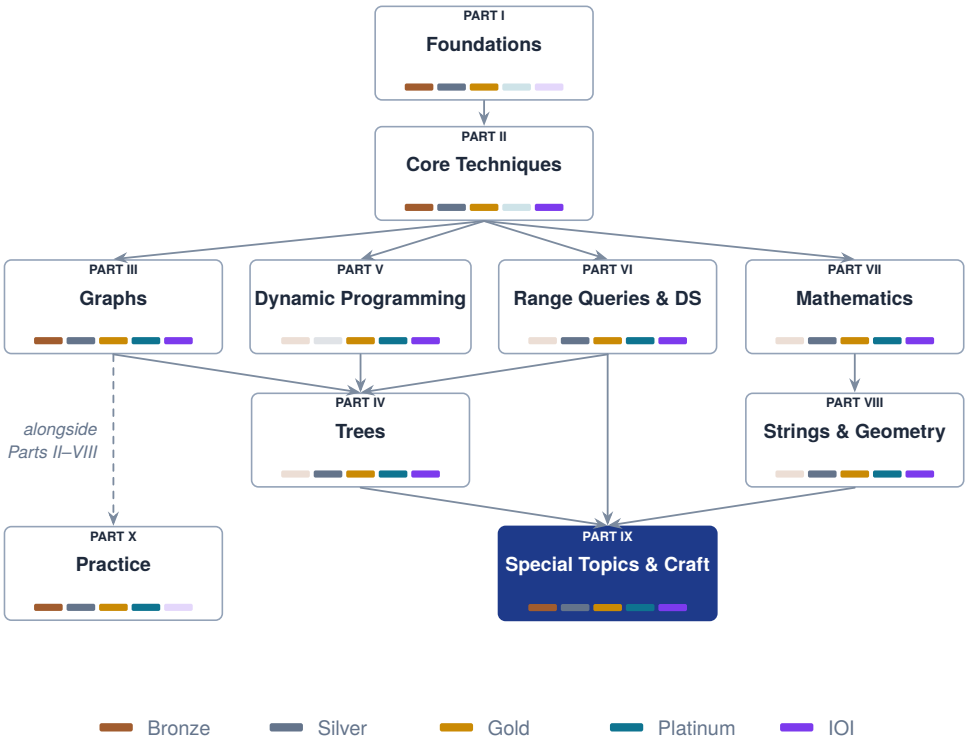
GEOMETRY

- Point Location Test
- Line Segment Intersection
- Polygon Area
- Point in Polygon
- Polygon Lattice Points
- Minimum Euclidean Distance
- Convex Hull
- Maximum Manhattan Distances
- All Manhattan Distances
- Intersection Points
- Line Segments Trace I
- Line Segments Trace II
- Lines and Queries I
- Lines and Queries II
- Area of Rectangles
- Robot Path

PART IX

Special Topics and Contest Craft

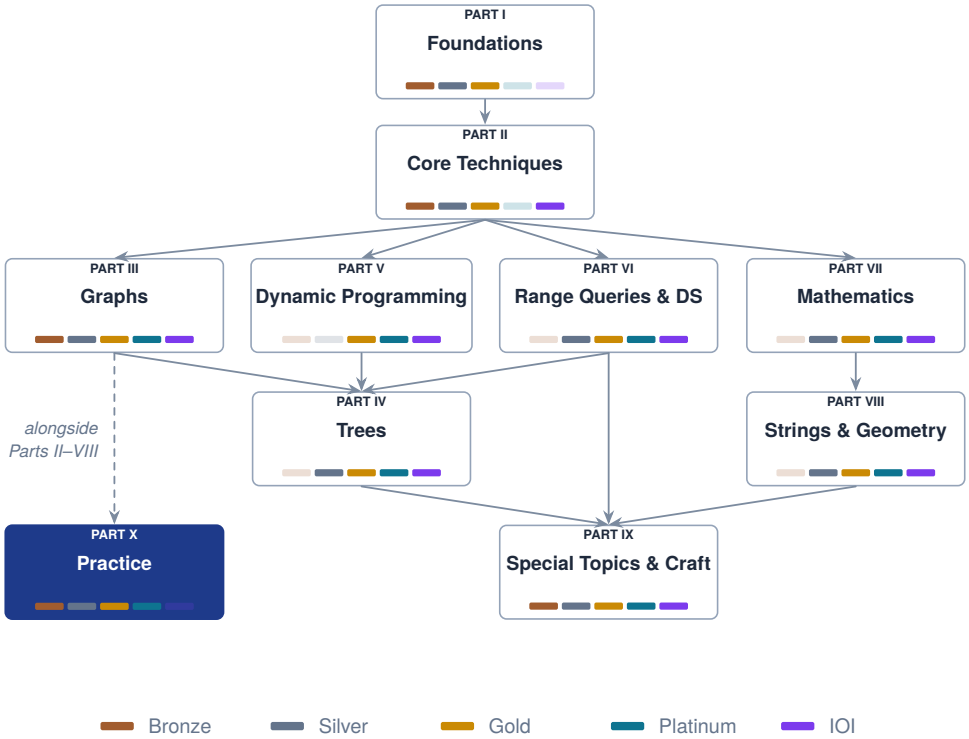
Randomization, interactive problems, bitsets, matroids and vectorization; the craft of contests; and a catalogue of the tricks experienced competitors reach for.



PART X

Practice

Classic problems solved end to end, from reading the limits to tested code. Use them alongside Parts II–VIII.



Index

Symbols

$O(f(n))$ . . . 30
0–1 BFS . . . 140
0/1 knapsack . . . 350
2-SAT . . . 181
2-edge-connected components . . . 185

A

A* . . . 149
adjacency list . . . 134
adjacency matrix . . . 134
AHU algorithm . . . 225
alternating path . . . 198
arc . . . 133
argsort . . . 81
articulation point . . . 183
assignment problem . . . 201
augmentation . . . 57
augmenting path . . . 193

B

Backtracking . . . 69
Bellman–Ford . . . 157
Berge's lemma . . . 198
BFS . . . 345
Biconnected components . . . 185
binary heap . . . 44
Binary lifting . . . 213
binary lifting . . . 358
binary search on the answer . . . 341
binary trie . . . 124
bipartite . . . 134, 142
bitmask . . . 66
block-cut tree . . . 185
branch and bound . . . 73
Breadth-first search . . . 137
Brent's algorithm . . . 146
bridge . . . 183
bridge tree . . . 185

C

center . . . 210
centroid . . . 210, 234
centroid tree . . . 234
children . . . 207
complete search . . . 64
condensation . . . 177
connected . . . 134
connected component . . . 343
connected components . . . 134
contribution technique . . . 14
convolution . . . 315
coordinate compression . . . 84
cut . . . 195
cycle . . . 134
cycle detection . . . 63

D

DAG . . . 134
data structure . . . 50
de Bruijn sequence . . . 128
degree . . . 134
depth . . . 207
Depth-first search . . . 135
deque . . . 53
DFS tree . . . 135
diameter . . . 209
difference array . . . 108
Dijkstra . . . 346
Dijkstra's algorithm . . . 152
Dilworth's theorem . . . 201
Dinkelbach's method . . . 101
disjoint set union . . . 165, 348
dominates . . . 191
dominator tree . . . 191
DSU on tree . . . 238
DSU with rollback . . . 188

E

edge list . . . 134
edges . . . 133
Edmonds' blossom algorithm . . . 203
Euler tour . . . 208
Euler tour technique . . . 210
Euler tour tree . . . 245

Euler's formula 190
Eulerian circuit 186
Eulerian path 186

F

factorial number system 69
Fenwick tree 356
Flood fill 140
flood fill 343
flow 193
flow network 193
Floyd's cycle-finding algorithm 146
Floyd–Warshall 161
forest 134
fractional programming 101
Fracturing search 75
functional graph 146, 358

G

Gomory–Hu tree 204
Gosper's hack 68
graph 133
Gray code 67, 127
greedy algorithm 85
Grundy number 312

H

half-open range 43
heap 54
heavy child 230
heavy paths 230
Heavy-light decomposition 230
height 241
Hierholzer's algorithm 186
Hopcroft–Karp 198
Huffman's algorithm 88
Hungarian algorithm 201

I

implication graph 181
inclusion–exclusion 17
invariant 20

K

Kadane's algorithm 337
Kahn's algorithm 174
Kosaraju's algorithm 179
Kruskal reconstruction tree 171
Kruskal's algorithm 169
Kuhn's algorithm 198

L

LCP array 320
Lehmer code 69
linear basis 314
link-cut tree 191, 246
long paths 241
longest increasing subsequence 258
losing 278
low-link 177
lowest common ancestor 213

M

master theorem 34
matching 198
matroid 328
Meet in the middle 73
Memoization 252
Menger's theorem 197
minimum spanning tree 168
minimum-cost flow 202
monotone predicate 96
monovariant 21
multi-source BFS 139

N

number theoretic transform 315

O

Offline processing 19

P

Parallel binary search 104
parent 207
path 134
path cover 200
pigeonhole principle 22
postorder 208
potential 33
prefix function 318, 357
preorder 208
Prim's algorithm 170
pruning 69
Prüfer code 172

Q

queue 53

R

radix sort 81
range query 295
recursion tree 34
reflection principle 23
Regret greedy 91
relaxation 152
Rerooting 220
residual graph 193

S

shortest-path tree 152
SMAWK 285
Smith's rule 83
spanning tree 168
stack 51
STL 39
Stoer–Wagner algorithm 204
stress test 11
strict weak ordering 84
strongly connected 177
strongly connected components 177
submask 67
subtree 207
suffix array 320
suffix automaton 321
suffix links 320
sum over subsets 272
sweep 19
system of difference constraints 159

T

Tabulation 252
Top trees 247
topological order 174
treap 303
tree 134
tree DP 353
trie 319
two pointers 338
two's complement 120

U

undefined behaviour 48
unimodal 102

V

vertices 133
virtual tree 243

W

weighted DSU 167
winning 278